



Minhaj
University
Lahore

Documentation

Compiler Construction Project

Project Title:

Infix to Postfix Converter

Subject: Compiler Construction

Submitted By:

Raffia Pervaiz (085)

Anam Shaheen (081)

Amna Mehmood (130)

Submitted To: Sir Rashid

Department: Computer Science

Semester / Section: 6th / E

Table Contents

- 1. Introduction3
- 2. Problem Statement.....3
- 3. Objectives3
- 4. Scope of the Project.....3
 - 5.1 Expression Notation Types.....3
 - 5.2 Operator Precedence Table.....3
 - 6.1 Algorithm Steps.....4
 - 6.2 Pseudocode4
 - Explanation of Each Step5
 - 8.1 Level 0 DFD (Context Diagram).....5
 - 8.2 Level 1 DFD (Detailed View)6
 - Components Explained.....6
 - Reading the Flowchart.....7
 - Step-by-Step Conversion.....8
 - Extended Example.....8
- 11. Test Cases9
- 13. Advantages9
- 14. Limitations.....9
- 15. Conclusion..... 10
- 16. Future Enhancements 10

1. Introduction

An Infix to Postfix Converter is a program that transforms an arithmetic expression written in infix notation into postfix notation (Reverse Polish Notation - RPN). Infix notation is the common mathematical format where operators are written between operands (e.g., $A + B$), while postfix notation places operators after operands (e.g., $A B +$).

In compiler construction, this conversion is an important step because postfix expressions can be evaluated easily using a stack without requiring operator precedence rules during execution. Many compilers internally convert expressions into postfix or similar intermediate forms before generating machine code.

2. Problem Statement

Arithmetic expressions written in infix form require handling of operator precedence and parentheses during evaluation. This increases the complexity of expression parsing. The goal of this project is to design and implement a system that converts infix expressions into postfix notation using a stack-based algorithm, making expression evaluation simpler and more efficient.

3. Objectives

- To understand the concept of infix and postfix expressions.
- To study the role of expression conversion in compiler design.
- To implement the stack data structure for operator management.
- To convert infix expressions into postfix expressions correctly.
- To demonstrate operator precedence and associativity handling.

4. Scope of the Project

The project accepts arithmetic expressions containing:

- Operands (A-Z, a-z, 0-9)
- Operators (+, -, *, /, ^)
- Parentheses ((and))

The system converts valid infix expressions into equivalent postfix expressions. The project can be extended in the future to support prefix conversion and direct expression evaluation.

5. Basic Concepts

5.1 Expression Notation Types

Expression Type	Example	Description
Infix	$A + B * C$	Operators sit between operands. This is what humans normally write.
Postfix	$A B C * +$	Operators come after operands. Also called Reverse Polish Notation (RPN).
Prefix	$+ A * B C$	Operators come before operands. Also called Polish Notation.

5.2 Operator Precedence Table

Operator	Priority
^	Highest
*, /	High
+, -	Low
(	Special Case

6. Algorithm (Infix to Postfix)

The Shunting Yard algorithm was devised by Edsger Dijkstra. It converts infix expressions to postfix in a single left-to-right pass, using one stack for operators and one queue for output.

6.1 Algorithm Steps

1. Start.
2. Read the infix expression from the user.
3. Initialize an empty stack.
4. Scan the expression from left to right.
5. If the scanned character is an operand, add it to the output.
6. If the scanned character is '(', push it onto the stack.
7. If the scanned character is ')', pop and output operators until '(' is encountered, then discard the '('.
8. If the scanned character is an operator: while the stack is not empty and the precedence of the top operator is greater than or equal to the current operator, pop operators from the stack and add them to the output. Then push the current operator onto the stack.
9. After scanning the complete expression, pop all remaining operators from the stack and append them to the output.
10. Display the postfix expression.
11. End.

6.2 Pseudocode

```
function infixToPostfix(tokens):
    stack <- empty
    output <- empty
    for each token in tokens:
        if isOperand(token):
            output.enqueue(token)
        elif token == '(':
            stack.push(token)
        elif token == ')':
            while stack.top() != '(':
                output.enqueue(stack.pop())
            stack.pop() // discard '('
        else: // operator
            while stack not empty and stack.top() != '('
                and prec(stack.top()) >= prec(token):
                output.enqueue(stack.pop())
            stack.push(token)
    while stack not empty:
```

```
output.enqueue(stack.pop())
return output
```

Note: Use strictly greater-than (>) instead of (>=) for right-associative operators such as ^ (exponentiation).

7. Workflow Diagram

The workflow diagram below shows the high-level steps the system follows when converting an infix expression to postfix from the moment the user enters an expression to when the postfix result is produced.

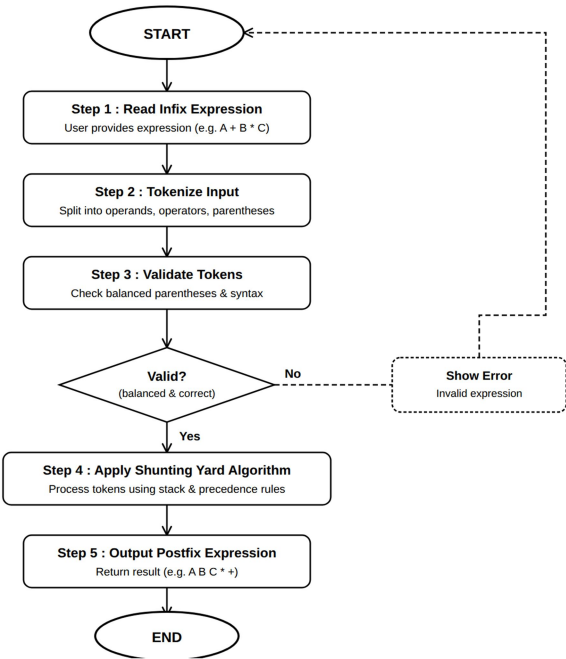


Figure 1 — Workflow Diagram: Infix to Postfix Conversion

Explanation of Each Step

- **Step 1 —Read Expression:** The user provides an infix expression such as $A + B * C$.
- **Step 2 — Tokenize:** The expression is split into individual tokens: operands, operators, and parentheses.
- **Step 3 — Validate:** The system checks that parentheses are balanced and the expression is syntactically correct.
- **Decision — Valid?:** If invalid, an error is shown and the user re-enters the expression. If valid, conversion proceeds.
- **Step 4 — Apply Shunting Yard:** The algorithm processes each token using a stack and precedence rules.
- **Step 5 — Output Postfix:** The resulting postfix expression is returned to the user.

8. Data Flow Diagram (DFD)

A Data Flow Diagram shows how data moves through the system: where it comes from, what processes transform it, and where it ends up.

8.1 Level 0 DFD (Context Diagram)

The Level 0 DFD, also called the Context Diagram, shows the entire system as a single process. It is the simplest possible view: one process box, the external user, and the two flows of data between them.

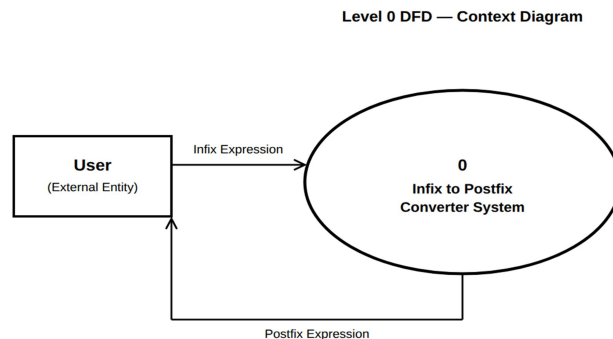


Figure 2 — Level 0 DFD: Context Diagram

8.2 Level 1 DFD (Detailed View)

The Level 1 DFD breaks the single process down into its three main internal processes, showing exactly how data flows between them and the two data stores they use.

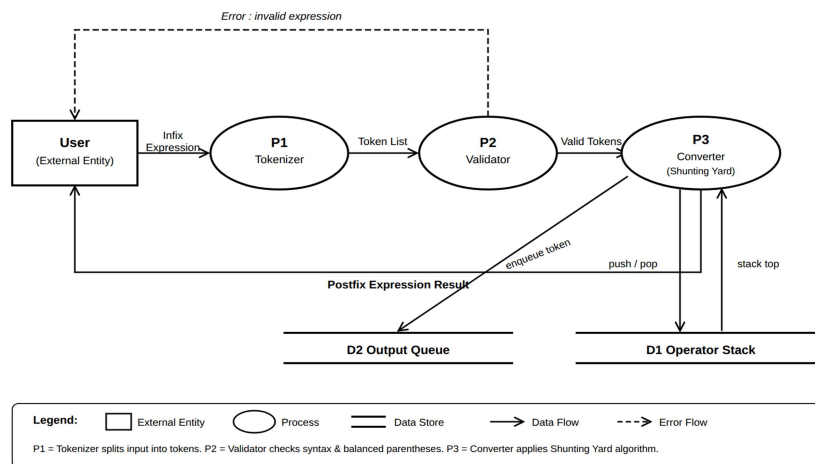


Figure 3 — Level 1 DFD: Infix to Postfix Converter

Components Explained

Symbol	Name	What it represents
Rectangle	External Entity	Something outside the system the User, who provides input and receives output.

Symbol	Name	What it represents
Oval / Circle	Process	A transformation step: P1 (Tokenizer), P2 (Validator), P3 (Converter).
Double Line	Data Store	Temporary storage during processing: D1 (Operator Stack) and D2 (Output Queue).
Solid Arrow	Data Flow	Movement of data between components, labelled with what is transferred.
Dashed Arrow	Error Flow	Error path sent back to the user when input is invalid.

9. Algorithm Flowchart

The flowchart below shows the detailed decision logic of the Shunting Yard algorithm exactly what happens for every type of token read from the input.

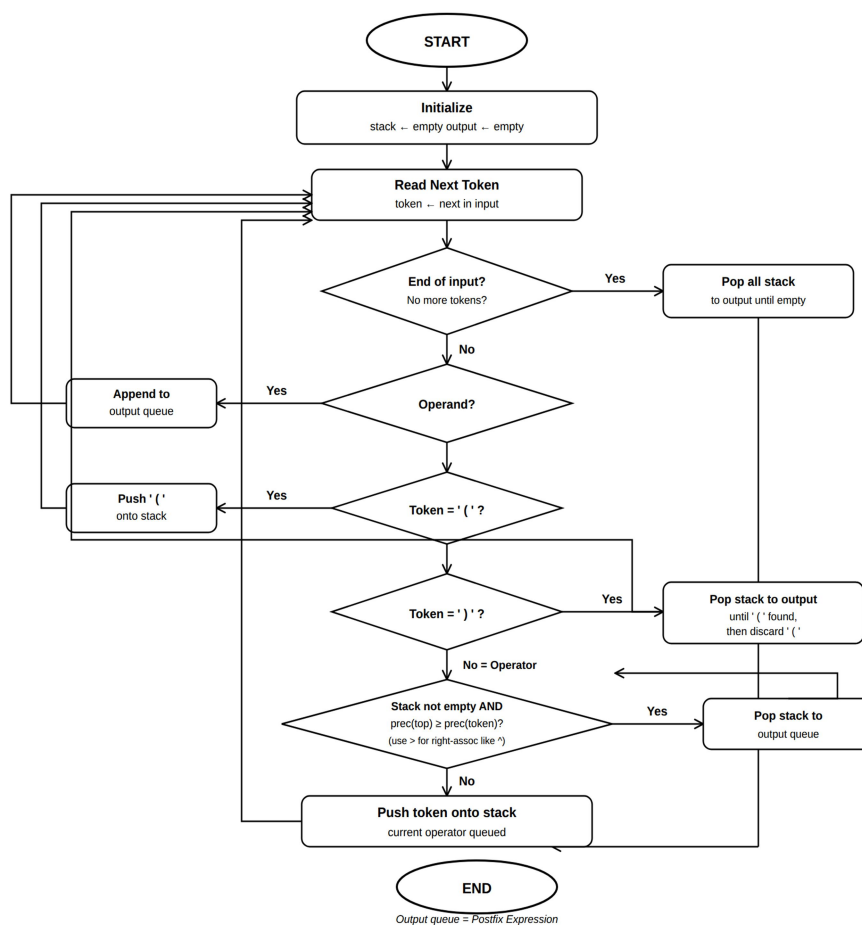


Figure 4 — Algorithm Flowchart: Shunting Yard (Infix to Postfix)

Reading the Flowchart

- Oval = Start / End terminal.
- Rectangle = Process (an action to perform).
- Diamond = Decision (a yes/no question that branches the flow).
- Dashed lines = error or loop-back paths returning control to an earlier step.

10. Worked Example

Input (Infix): $A + B * C - D$

Step-by-Step Conversion

Symbol	Stack	Output
A	Empty	A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
-	-	A B C * +
D	-	A B C * + D
End	Empty	A B C * + D -

Output (Postfix): $A B C * + D -$

Extended Example

To further demonstrate mixed precedence and right-associativity, consider: $A + B * C - D / E$

Step	Token	Action	Operator Stack	Output Queue
1	A	Operand -> output		A
2	+	Stack empty -> push +	+	A
3	B	Operand -> output	+	A B
4	*	$\text{prec}(*) > \text{prec}(+) \rightarrow \text{push } *$	+ *	A B
5	C	Operand -> output	+ *	A B C
6	-	Pop * (higher), pop + (equal), push -	-	A B C * +
7	D	Operand -> output	-	A B C * + D
8	/	$\text{prec}(/) > \text{prec}(-) \rightarrow \text{push } /$	- /	A B C * + D
9	E	Operand -> output	- /	A B C * + D E
10	(end)	Pop / then - to output		A B C * + D E / -

Final postfix result: $A B C * + D E / -$

11. Test Cases

The following test cases verify the algorithm handles common patterns correctly:

Infix Expression	Expected Postfix	What is tested
$A + B$	$A B +$	Basic addition
$A + B * C$	$A B C * +$	$*$ has higher precedence than $+$
$(A + B) * C$	$A B + C *$	Parentheses override precedence
$A * B + C * D$	$A B * C D * +$	Same-level operators, left-to-right
$A + B * C - D / E$	$A B C * + D E / -$	Mixed precedence levels
$A \wedge B \wedge C$	$A B C \wedge \wedge$	Right-associativity of $\wedge$
$((A + B) * C) - D$	$A B + C * D -$	Nested parentheses

12. Applications

- Compiler Design and Construction.
- Expression Parsing.
- Intermediate Code Generation.
- Calculator Applications.
- Stack-based Expression Evaluation.
- Virtual Machine and Interpreter Design.

13. Advantages

- Simplifies expression evaluation.
- Eliminates the need for parentheses.
- Easy implementation using stacks.
- Efficient for compilers and interpreters.
- Reduces complexity during code generation.

14. Limitations

- Assumes the input expression is syntactically correct.
- Does not handle invalid symbols unless additional validation is implemented.
- Basic version supports only arithmetic operators.

15. Conclusion

The Infix to Postfix Converter is an essential component in compiler construction because it transforms complex infix expressions into a format that is easier for machines to process. By using a stack-based algorithm, the converter correctly handles operator precedence and associativity.

This project strengthens the understanding of parsing techniques, stack operations, and intermediate code generation, making it an important educational exercise in compiler design.

Key takeaways from this project:

- Postfix notation eliminates the need for parentheses and precedence rules during evaluation.
- The Shunting Yard algorithm solves the problem in $O(n)$ time using a single stack.
- Operator precedence and associativity determine exactly when to pop from the stack.
- The DFD shows the data movement; the flowchart shows the decision logic; the workflow diagram shows the overall process.

16. Future Enhancements

- Add infix to prefix conversion.
- Evaluate postfix expressions automatically.
- Support floating-point and multi-digit operands.
- Detect and report syntax errors.
- Integrate with a simple compiler front-end.